Chapter 06: Function Templates

ENG3091

Khiem Nguyen

Contents

1 Why templates? 2
- 1.1 You've already used templates . . . 2
- 1.2 One function per type? . . . 2
- 1.3 A wish . . . 2

2 Function templates 3
- 2.1 Writing a function template . . . 3
- 2.2 Naming template parameters . . . 3
- 2.3 Using a function template . . . 3
- 2.4 Function template instantiation . . . 4
- 2.5 What the compiler generates . . . 4
- 2.6 Instantiated once, then reused . . . 5
- 2.7 …which is the same as . . . 6

3 Template argument deduction 6
- 3.1 Let the compiler work out `T` . . . 6
- 3.2 Templates versus ordinary functions . . . 7
- 3.3 Why that's useful . . . 8

4 More about function templates 8
- 4.1 Templates can have ordinary parameters too . . . 8
- 4.2 Some instances don't compile . . . 9
- 4.3 Some instances compile, but make no sense . . . 10
- 4.4 Templates live in header files . . . 10

5 Templates with several types 11
- 5.1 `max(2, 3.5)` doesn't compile . . . 11
- 5.2 Fix 1: say which type you want . . . 11
- 5.3 Fix 2: two type parameters? . . . 12
- 5.4 Fix 3: let the compiler choose the return type . . . 12
- 5.5 Or say it with `std::common_type_t` . . . 13
- 5.6 The `auto` keyword . . . 13
- 5.7 Abbreviated function templates (C++20) . . . 14
- 5.8 Function templates can be overloaded . . . 14

6 Non-type template parameters 15
- 6.1 `constexpr` functions . . . 15
- 6.2 Non-type template parameters . . . 15
- 6.3 A non-type template parameter in action . . . 16
- 6.4 The compiler's point of view . . . 16

7 Wrapping up 17
7.1 Summary . . . 17
7.2 Keep learning . . . 17

1 Why templates?

1.1 You've already used templates

In the lab assignments you met types like these:

- `std::vector<double>`: a growable list of `double`s;
- `std::vector<int>`: the same thing, holding `int`s.

And functions like these:

- `std::max(1, 2)`: the compiler works out that both arguments are `int`;
- `std::max<double>(10, 42.24)`: we tell it to treat both as `double`.

The part in angle brackets `< >` picks the type. `std::vector` and `std::max` are **templates**: written once, usable with many types. Today we learn to write our own **function templates**.

1.2 One function per type?

```
int max(int x, int y) {
    return (x < y) ? y : x;
}

double max(double x, double y) { // overloading, Chapter 05
    return (x < y) ? y : x;
}
```

`c ? a : b` is the **conditional operator**: "if `c` then `a`, otherwise `b`", like Python's `a if c else b`.

Two copies of the same code already. And `long`, `float`, `char`…? Every new type means another copy. C++ has a feature designed for exactly this problem: **templates**.

1.3 A wish

What we'd like to write, once:

```
T max(T x, T y) { // error: what is T?
    return (x < y) ? y : x;
}
```

…where `T` stands for *any* type for which `x < y` makes sense.

A **template** gives us exactly that:

- one definition, with a **placeholder type** `T` that isn't known yet;
- the compiler then **generates** a real function for each type we actually use.

2 Function templates

2.1 Writing a function template

```
template <typename T> // the template parameter declaration
T max(T x, T y) {     // the function template itself
    return (x < y) ? y : x;
}
```

- The **template parameter declaration** starts with the keyword `template`, then lists the placeholder types in angle brackets.
- `typename T` declares `T` as a **type template parameter**. (`class T` also works, but `typename` is preferred.)
- This whole definition is a **function template**: a recipe the compiler uses to generate functions, one per type.

2.2 Naming template parameters

- If the meaning is obvious, use a single capital letter: `T`, then `U`, `V`, …
- If the parameter has a specific job, give it a descriptive name starting with a capital letter, like `Allocator` (the standard library's style) or `TAllocator`.

Any name works as far as the compiler is concerned:

```
template <typename Banana>
Banana max(Banana x, Banana y) { // legal, but please don't
    return (x < y) ? y : x;
}
```

> Best practice
>
> Use `T`, `U`, `V`… for type template parameters that are used in obvious ways.

2.3 Using a function template

Put the actual type in angle brackets: `max<int>(1, 2)`. The type in the brackets is the **template argument**.

Listing 1 `use_template_function.cpp`

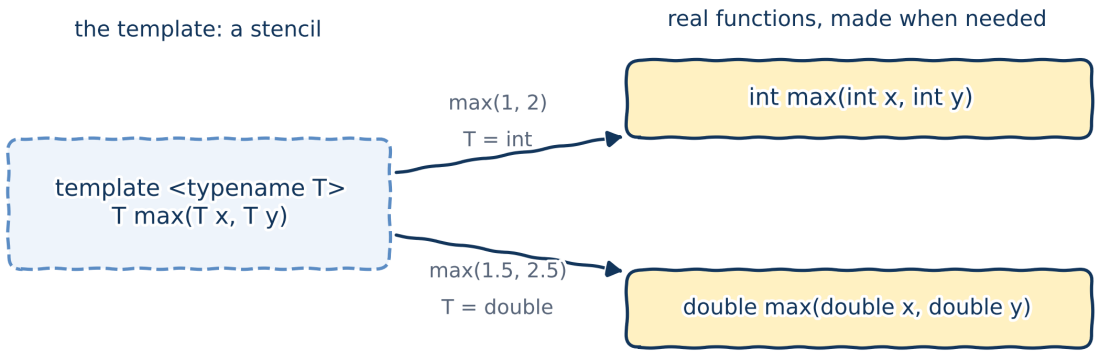
```
#include <iostream>

template <typename T>
T max(T x, T y) {
    return (x < y) ? y : x;
}

int main() {
    std::cout << max<int>(1, 2) << '\n';         // instantiates max<int>
    std::cout << max<double>(1.5, 0.5) << '\n'; // instantiates max<double>
    return 0;
}
```

```
2
1.5
```

2.4 Function template instantiation



- Creating a real function from a template is called **instantiation**. It happens when a call needs it (**implicit instantiation**).
- The generated function is a **function instance** (technically, a *specialization*). The template it came from is the **primary template**.
- Function instances are ordinary functions in every way.

> **!** Key idea
>
> A function template is **not** a function. It's a recipe for making functions.

2.5 What the compiler generates

When it meets `max<int>(1, 2)`, the compiler copies the template, replaces every `T` with `int`, and compiles the result. As if we'd written:

Listing 2 `instantiation_look_breakdown.cpp`

```cpp
#include <iostream>

// Only a declaration of the template is left...
template <typename T>
T max(T x, T y);

// ...plus the function the compiler generated from it for T = int
template <>
int max<int>(int x, int y) {
    return (x < y) ? y : x;
}

int main() {
    std::cout << max<int>(1, 2) << '\n';
    return 0;
}
```

You'll never write `template <>` like this yourself in this course: it's only here to show you what the compiler produces. (It compiles, and prints 2.)

2.6 Instantiated once, then reused

Listing 3 `instantiation_look_more.cpp`

```cpp
#include <iostream>

template <typename T>
T max(T x, T y) {
    return (x < y) ? y : x;
}

int main() {
    std::cout << max<int>(1, 2) << '\n';    // instantiates max<int>
    std::cout << max<int>(4, 3) << '\n';    // reuses max<int>
    std::cout << max<double>(1, 2) << '\n'; // instantiates max<double>
    return 0;
}
```

```
2
4
2
```

- A template is instantiated the **first** time a call needs that type. Later calls reuse the same function.
- A template that's never called is never instantiated: no function is generated at all.

2.7 ...which is the same as

Listing 4 `instantiation_look_more_breakdown.cpp`

```cpp
#include <iostream>

template <typename T>
T max(T x, T y); // only the declaration is left

template <>
int max<int>(int x, int y) { // generated for T = int
    return (x < y) ? y : x;
}

template <>
double max<double>(double x, double y) { // generated for T = double
    return (x < y) ? y : x;
}

int main() {
    std::cout << max<int>(1, 2) << '\n';
    std::cout << max<int>(4, 3) << '\n';
    std::cout << max<double>(1, 2) << '\n';
    return 0;
}
```

Two instances: `max<int>` (used twice) and `max<double>`. Note that `max<double>(1, 2)` returns `2.0`, which `std::cout` prints as 2.

3 Template argument deduction

3.1 Let the compiler work out `T`

Usually, the template argument just repeats the types of the arguments:

```cpp
max<int>(1, 2) // we say T = int... but 1 and 2 are ints anyway
```

So we can leave it out, and let the compiler **deduce** `T` from the arguments:

```cpp
max<>(1, 2) // T deduced as int; only templates are considered
max(1, 2)   // T deduced as int; ordinary functions are considered too
```

The second form looks exactly like an ordinary function call, and that's the one we normally use.

3.2 Templates versus ordinary functions

Listing 5 `template_argument_deduction_more.cpp`

```cpp
#include <iostream>

template <typename T>
T max(T x, T y) {
    std::cout << "template max<T>: ";
    return (x < y) ? y : x;
}

int max(int x, int y) {
    std::cout << "ordinary max: ";
    return (x < y) ? y : x;
}

int main() {
    std::cout << max<int>(1, 2) << '\n'; // template, T = int
    std::cout << max<>(1, 2) << '\n';    // template, T deduced
    std::cout << max(1, 2) << '\n';      // ordinary wins
    return 0;
}
```

```
template max<T>: 2
template max<T>: 2
ordinary max: 2
```

> **!** Key idea
>
> When an ordinary function and a template instance match equally well, `max(1, 2)` picks the **ordinary function**.

3.3 Why that's useful

Listing 6 `favor_normal_func_over_func_template_call.cpp`

```cpp
#include <iostream>

template <typename T>
void print(T x) { // generic: any T
    std::cout << x << '\n';
}

void print(bool x) { // special: bool
    std::cout << std::boolalpha << x << '\n';
}

int main() {
    print<bool>(true); // template: 1
    print<>(true);     // template: 1
    print(true);       // ordinary: true
    return 0;
}
```

```
1
1
true
```

> **Best practice**
>
> Call function templates with the normal syntax, `print(true)`, unless you specifically need the template version.

The template handles every type in a general way; the ordinary function handles `bool` specially. With the normal call syntax, the special version wins automatically.

4 More about function templates

4.1 Templates can have ordinary parameters too

A function template can mix template parameters with ordinary ones, and those can have default arguments (Chapter 05):

Listing 7 `func_template_with_non_template_parameters.cpp`

```cpp
#include <iostream>

// T: template parameter; times: an ordinary int
template <typename T>
void print_repeated(T value, int times = 1) {
    for (int i{0}; i < times; ++i) {
        std::cout << value << ' ';
    }
    std::cout << '\n';
}

int main() {
    print_repeated(5);      // T = int, times = 1
    print_repeated('a', 3); // T = char, times = 3
    print_repeated(2.5, 2); // T = double, times = 2
    return 0;
}
```

```
5
a a a
2.5 2.5
```

4.2 Some instances don't compile

Listing 8 `instantiated_func_not_compile.cpp`

```cpp
#include <iostream>
#include <string>

template <typename T>
T add_value(T x, double v) {
    return x + v;
}

int main() {
    std::cout << add_value(4.2, 2.4) << '\n'; // fine: double + double

    std::string hello{"Hello, world!"};
    std::cout << add_value(hello, 4.2) << '\n'; // string + double?!
    return 0;
}
```

```
error: no match for 'operator+' (operand types are 'std::__cxx11::basic_string<char>' and 'double')
```

...followed by dozens of lines of notes. The template itself is fine; the **instance** `add_value<std::string>` isn't, because you can't add a `double` to a string. Template error messages are famously long: read the first line first!

4.3 Some instances compile, but make no sense

Listing 9 `not_make_sense_semantically.cpp`

```cpp
#include <iostream>

template <typename T>
T add_one(T x) {
    return x + 1;
}

int main() {
    std::cout << add_one(41) << '\n';               // fine: 42
    std::cout << add_one("Hello, world!") << '\n'; // compiles... but?
    return 0;
}
```

```
42
ello, world!
```

`"Hello, world!"` is not a `std::string`: it's a *C-style string*, and `T` becomes the memory address of its first character. Adding 1 moves to the next character, so the `H` is lost. (Addresses are the topic of Chapter 08.) The compiler can't know that wasn't what you meant.

4.4 Templates live in header files

Listing 10 `max.h`

```cpp
#ifndef MAX_H
#define MAX_H

// The whole template lives here
template <typename T>
T max(T x, T y) {
    return (x < y) ? y : x;
}

#endif
```

Listing 11 `main.cpp`

```cpp
#include "max.h"
#include <iostream>

int main() {
    std::cout << max(3, 7) << '\n';
    std::cout << max(2.5, 1.5) << '\n';
    return 0;
}
```

```
7
2.5
```

- `main.cpp` instantiates `max<int>` and `max<double>`. To do that, the compiler needs the template's **whole definition**, body included, in every `.cpp` file that uses it. A declaration alone isn't enough.
- So templates go **entirely in the header**. They are exempt from the "define exactly once" rule. This is the exception to the header rules from Chapter 02.

5 Templates with several types

5.1 `max(2, 3.5)` doesn't compile

Listing 12 `max_mismatched_types.cpp`

```cpp
#include <iostream>

template <typename T>
T max(T x, T y) {
    return (x < y) ? y : x;
}

int main() {
    std::cout << max(2, 3.5) << '\n'; // int and double: what is T?
    return 0;
}
```

```
error: no matching function for call to 'max(int, double)'
note:   deduced conflicting types for parameter 'T' ('int' and 'double')
```

Can somebody explain why? (Hint: read the second line.)

`T` can only be one type. From `2` the compiler deduces `int`, from `3.5` it deduces `double`, and it refuses to choose.

5.2 Fix 1: say which type you want

Listing 13 `max_explicit_type.cpp`

```cpp
#include <iostream>

template <typename T>
T max(T x, T y) {
    return (x < y) ? y : x;
}

int main() {
    std::cout << max<double>(2, 3.5) << '\n'; // T = double: 2 becomes 2.0
    return 0;
}
```

```
3.5
```

With `T` given explicitly, there's nothing to deduce: `2` is simply converted to `2.0`. It works, but it would be nicer not to have to think about the types at all.

5.3 Fix 2: two type parameters?

Listing 14 `two_template_type_parameters.cpp`

```cpp
#include <iostream>

template <typename T, typename U> // two type template parameters
T max(T x, U y) {                 // x has type T, y has type U
    return (x < y) ? y : x;       // the result is converted to T!
}

int main() {
    std::cout << max(2, 3.5) << '\n'; // T = int, U = double
    return 0;
}
```

Now `x` and `y` can have different types: `T = int`, `U = double`. Let's run it!

```
3
```

Why `3`, not `3.5`? The **return type** is `T`, which is `int`, so the result `3.5` is converted to `int`, losing the `.5`. No error, no warning: just a wrong answer.

5.4 Fix 3: let the compiler choose the return type

Listing 15 `two_template_type_parameters_auto.cpp`

```cpp
#include <iostream>

template <typename T, typename U>
auto max(T x, U y) { // let the compiler work out the return type
    return (x < y) ? y : x;
}

int main() {
    std::cout << max(2, 3.5) << '\n';
    return 0;
}
```

```
3.5
```

With the return type `auto`, the compiler works it out from the `return` statement. Since the conditional operator's two branches have types `int` and `double`, the result is a `double`.

5.5 Or say it with `std::common_type_t`

`std::common_type_t<T, U>` is the type that both `T` and `U` convert to without losing information:

`T` and `U`	Common type
`int`, `double`	`double`
`int`, `long`	`long`
`float`, `double`	`double`

Listing 16 `using_common_type_t.cpp`

```cpp
#include <iostream>
#include <type_traits> // common_type_t

template <typename T, typename U>
std::common_type_t<T, U> max(T x, U y) {
    return (x < y) ? y : x;
}

int main() {
    std::cout << max(5, 10.5) << '\n';
    return 0;
}
```

```
10.5
```

It needs `#include <type_traits>`. With modern compilers, `auto` is usually simpler.

5.6 The `auto` keyword

`auto` asks the compiler to **deduce a type** from the initializer:

```cpp
auto age{25};           // int
auto price{42.99};      // double
auto initial{'K'};      // char
auto name{std::string{"Khiem"}}; // std::string
```

- This does **not** make C++ dynamically typed like Python. Each variable still gets exactly one type, fixed at compile time; you just don't have to spell it out.
- Careful: plain `auto` drops `const`. Write `const auto` if you want to keep it. (`auto` and references: next chapter.)

5.7 Abbreviated function templates (C++20)

Listing 17 `abbreviated_function_template.cpp`

```cpp
#include <iostream>

auto max(auto x, auto y) { // C++20: a template without the template line
    return (x < y) ? y : x;
}

int main() {
    std::cout << max(2, 3.5) << '\n';   // int and double
    std::cout << max('a', 'z') << '\n'; // char and char
    return 0;
}
```

```
3.5
z
```

Since C++20, a parameter of type `auto` turns an ordinary function into a function template, with one type template parameter per `auto`. The code above is shorthand for `template <typename T, typename U> auto max(T x, U y)`.

5.8 Function templates can be overloaded

Listing 18 `function_templates_overloaded.cpp`

```cpp
#include <iostream>

template <typename T>
auto add(T x, T y) { // same types
    return x + y;
}

template <typename T, typename U>
auto add(T x, U y) { // different types
    return x + y;
}

template <typename T, typename U, typename V>
auto add(T x, U y, V z) { // three values
    return x + y + z;
}

int main() {
    std::cout << add(1.2, 3.4) << '\n'; // add<double>
    std::cout << add(5.6, 7) << '\n';   // add<double, int>
    std::cout << add(8, 9, 10) << '\n'; // add<int, int, int>
    return 0;
}
```

```
4.6
12.6
27
```

6 Non-type template parameters

6.1 `constexpr` functions

Remember `constexpr` variables from Chapter 04? A function can be `constexpr` too, so that the compiler can call it **while compiling**:

```
constexpr int square(int x) {
    return x * x;
}

constexpr int result{square(10)}; // 100, worked out at compile time
```

We need this idea for the next kind of template parameter.

6.2 Non-type template parameters

A template parameter doesn't have to be a type. A **non-type template parameter** has a fixed type and stands for a **value** known at compile time.

It can be, for example:

- an integer type: `int`, `char`, `bool`, …
- a floating-point type (since C++20);
- an enumeration (later in the course);
- a pointer or reference (also later).

6.3 A non-type template parameter in action

Listing 19 `nontype_template_parameters.cpp`

```cpp
#include <iostream>

// Factor is a value, not a type
template <int Factor, typename T>
T scale_sensor_reading(T raw_value) {
    return raw_value * Factor;
}

int main() {
    int raw_input{10};

    int high{scale_sensor_reading<5>(raw_input)}; // Factor = 5
    int low{scale_sensor_reading<2>(raw_input)};  // Factor = 2

    std::cout << "Raw: " << raw_input << '\n';
    std::cout << "High (x5): " << high << '\n';
    std::cout << "Low  (x2): " << low << '\n';
    return 0;
}
```

```
Raw: 10
High (x5): 50
Low  (x2): 20
```

`Factor` is given explicitly in the angle brackets; `T` is still deduced from the argument.

6.4 The compiler's point of view

Each different `Factor` produces a different function, with the value baked in:

Listing 20 `nontype_template_parameters_pov.cpp`

```cpp
#include <iostream>

template <int Factor, typename T>
T scale_sensor_reading(T raw_value); // declaration only

template <>
int scale_sensor_reading<5>(int raw_value) { // Factor = 5
    return raw_value * 5;
}

template <>
int scale_sensor_reading<2>(int raw_value) { // Factor = 2
    return raw_value * 2;
}

int main() {
    int raw_input{10};
    std::cout << scale_sensor_reading<5>(raw_input) << '\n';
    std::cout << scale_sensor_reading<2>(raw_input) << '\n';
    return 0;
}
```

```
50
20
```

7 Wrapping up

7.1 Summary

- A **function template** is a recipe for functions: `template <typename T>` introduces a placeholder type `T`.
- The compiler **instantiates** a real function for each type a call needs. The template itself is not a function.
- **Template argument deduction** works out `T` from the arguments: write `max(1, 2)`, not `max<int>(1, 2)`. An equally good ordinary function wins.
- An instance may fail to compile, or compile and do something silly: the compiler only checks it once it's instantiated.
- Templates live **entirely in header files**.
- For mixed types, use two type parameters with an `auto` return type, or the C++20 shorthand `auto max(auto x, auto y)`.
- A **non-type template parameter** is a compile-time value, like `template <int Factor>`.

7.2 Keep learning

Explore these on your own, with Google Gemini in Guided Learning mode or on learncpp.com:

1. Function templates with multiple template types
2. Non-type template parameters
3. Type deduction for objects using `auto`
4. Constexpr functions

Next up, Chapter 07: references, and how to pass arguments without copying them.